Unit Test Documentation by Rizky Hardiana

1. Module
- Module Name: Material Registry Management
- Model: material.registry

2. Purpose of Testing
Melakukan validasi fitur utama pada model material.registry, untuk memastikan:
- Data material dapat dibuat dengan valid.
- Validasi harga beli (buy_price) minimal berfungsi.
- Constraint kode material (code) bersifat unik.

3. Test Environment
- Framework: Odoo 14
- Testing Class: TransactionCase
- Database: Testing Database (isolated transactions)

4. Test Cases Matrix

No	Test Case	Input Data	Expected Result
1	Create valid material	code: MAT001, name: Material A, type: fabric, buy_price: 150, supplier_id: valid ID	Material berhasil dibuat. Semua field sesuai.
2	Validate buy_price < 100	code: MAT002, name: Material B, type: jeans, buy_price: 50, supplier_id: valid ID	Raise ValidationError ("Material Buy Price must be at least 100.")
3	Validate unique constraint code	2 records dengan code yang sama: MAT003	Raise ValidationError ("Material code must be unique.")

5. Test Functions

```
from odoo.tests.common import TransactionCase
from odoo.exceptions import ValidationError

class TestMaterialRegistry(TransactionCase):

    def setUp(self):
        super().setUp()
        self.supplier = self.env['res.partner'].create({
            'name': 'Test Supplier',
            'supplier_rank': 1,
        })

    def test_create_valid_material(self):
        material = self.env['material.registry'].create({
            'code': 'MAT001',
            'name': 'Material A',
            'type': 'fabric',
            'buy_price': 150,
            'supplier_id': self.supplier.id,
        })
        self.assertEqual(material.code, 'MAT001')
        self.assertEqual(material.name, 'Material A')
        self.assertEqual(material.type, 'fabric')
        self.assertEqual(material.buy_price, 150)

    def test_buy_price_validation(self):
        with self.assertRaises(ValidationError) as context:
            self.env['material.registry'].create({
                'code': 'MAT002',
                'name': 'Material B',
                'type': 'jeans',
                'buy_price': 50,
                'supplier_id': self.supplier.id,
            })
        self.assertIn('Material Buy Price must be at least 100.', str(context.exception))

    def test_unique_code_constraint(self):
        self.env['material.registry'].create({
            'code': 'MAT003',
            'name': 'Material C',
            'type': 'cotton',
            'buy_price': 200,
            'supplier_id': self.supplier.id,
        })
        with self.assertRaises(ValidationError):
```

```
self.env['material.registry'].create({
    'code': 'MAT003',
    'name': 'Material D',
    'type': 'cotton',
    'buy_price': 200,
    'supplier_id': self.supplier.id,
})
```

6. How to Run Unit Test

Command to execute only this module's test:
python odoo-bin -c <path_config> -d <database_name> -u kedatech_registry_material_test_custom_modul --test-enable --stop-after-init

7. Notes

- Semua transaksi test akan otomatis rollback (data tidak akan tersimpan di database utama).
- Untuk testing REST API sudah dilakukan hasilnya ada di folder.
- Coverage testing minimal sudah mencakup 3 critical path.

Document Version
- Tanggal: 1 Mei 2025
- Author: Rizky Hardiana
- Version: v1.0